

项目网络请求架构分析

以下是按 Word 文档标准排版的内容（直接复制到 Word 中即可使用，保留标题层级、列表结构、代码格式，无额外添加文字）：

项目网络请求架构分析

一、整体架构概览

项目网络请求采用“分层设计模式”，将复杂的网络操作拆分为三个独立层级，职责清晰、易于维护，核心架构如下：

- 底层**：使用 Alamofire 库执行实际的网络请求（发送 HTTP 请求、接收服务器响应），是网络操作的“基础引擎”；
- 中间层**：NetworkManager（网络管理器）封装基础请求方法（GET/POST/ 文件上传），统一处理请求头、超时配置与网络错误；
- 业务层**：BaseAPI（基础 API 工具类）与 CommonApiObj（业务 API 对象）协同，处理业务相关的 API 调用、数据格式转换。

二、核心组件详解

1. 网络请求管理器（NetworkManager）

NetworkManager 是网络请求的“中枢”，采用 **单例模式**（全 App 仅一个实例），确保网络配置统一、避免资源浪费。

核心代码（单例初始化）

```
// 单例模式，全局只有一个实例
class NetworkManager {
    static let shared = NetworkManager()
    private init() {} // 防止外部创建新实例，确保单例唯一性
}
```

核心作用

- 封装 GET、POST 请求与文件上传逻辑，隐藏 Alamofire 底层调用细节；
- 自动添加通用请求头（如用户 Token、Content-Type: application/json），无需重复配置；
- 统一处理网络错误（如断网、超时），将技术错误转换为用户友好的提示信息。

2. 基础 API 类（BaseAPI）

BaseAPI 是 **抽象工具类**（仅提供静态方法，不初始化实例），是连接 NetworkManager 与业务层的“桥梁”。

核心作用

- 调用 NetworkManager 发起网络请求，传递业务层参数；
- 将服务器返回的 JSON 数据，解析为遵循 Decodable 协议的模型对象（如 User、Meeting）；
- 将业务层的模型对象，转换为网络请求所需的字典参数（[String: Any]）。

3. 业务 API 对象（CommonApiObj）

CommonApiObj 是 **面向具体业务模块的封装**，每个业务模块（如用户管理、会议管理、消息通知）可创建专属实例，专注本模块 API 逻辑。

核心作用

- 封装业务模块的 API 路径（如用户列表路径 “/api/user/list”），避免硬编码；
- 提供“增删改查”等业务方法（如 list () 获取列表、create () 创建数据）；
- 处理业务专属参数（如分页页码、筛选条件），返回业务相关的模型数据。

4. 视图模型（CommonVM）

CommonVM（ViewModel）是 **UI 层与业务层的桥梁**，隔离 UI 与网络逻辑，负责数据流转与状态管理。

核心作用

- 接收 UI 层请求，调用 CommonApiObj 获取数据；
- 处理数据（如过滤、排序），更新 UI 状态（如 isLoading 加载中、errorMessage 错误提示）；
- 以 @Published 修饰数据（如 userList 数组），供 UI 层监听，触发自动刷新。

三、网络请求流程

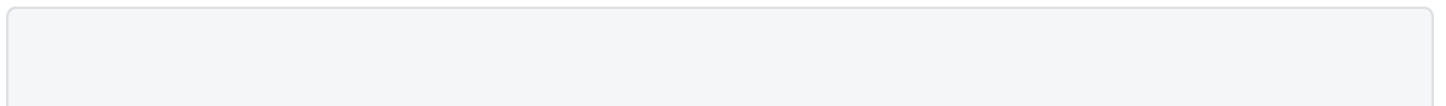
以“获取用户列表”为例，完整展示数据从“UI 层→网络→UI 层”的流转，共 6 步：

1. **UI 层发起请求**：UserListView（用户列表界面）调用 UserVM（用户视图模型）的 fetchUserList (page: 1, pageSize: 10) 方法；
2. **ViewModel 调用业务 API**：UserVM 内部创建 CommonApiObj（用户业务 API 对象），调用其 list (parameters: ["page": 1, "pageSize": 10]) 方法；
3. **业务 API 调用基础 API**：CommonApiObj 的 list () 方法，调用 BaseAPI 的 getList (path: "/api/user/list", parameters: ...) 静态方法；
4. **基础 API 调用网络管理器**：BaseAPI 调用 NetworkManager.shared.requestPost (path: 完整路径, - parameters: ...) 方法，发起 POST 请求；
5. **网络管理器执行请求**：NetworkManager 使用 Alamofire 发送 HTTP 请求，携带通用请求头，等待服务器响应；
6. **数据返回与 UI 更新**：
 - 服务器响应后，数据按“NetworkManager→BaseAPI→CommonApiObj→CommonVM→UI 层”返回；
 - CommonVM 更新数据与状态（如 userList = 解析后的用户数组、isLoading = false）；
 - UI 层监听 CommonVM 状态变化，自动刷新界面，显示用户列表。

四、关键代码解析

1. 发送 GET 请求（NetworkManager）

封装 GET 请求，处理 URL 拼接、参数、请求头与结果回调：



```

// NetworkManager中的GET请求封装
func requestGet(path: String, parameters: Parameters?, completion:
@escaping NetworkRequestCompletion) -> DataRequest {
    // 拼接URL，添加参数和请求头，设置超时时间
    AF.request(NetworkAPIBaseURL + path,
        parameters: parameters,
        headers: commonHeaders,
        requestModifier: { $0.timeoutInterval = 30 })
    .responseData { response in
        // 处理请求结果
        switch response.result {
        case let .success(data): completion(.success(data))
        case let .failure(error): completion(self.handleError
(error))
        }
    }
}

```

代码说明

- 用 Alamofire 发送 GET 请求，URL = 基础 URL + 接口路径；
- 自动携带 commonHeaders（含 Token、Content-Type），超时时间 30 秒；
- 请求完成后，通过 completion 回调返回“成功数据”或“处理后的错误”。

2. 处理网络错误（NetworkManager）

将 Alamofire 原始错误转换为用户友好提示，统一错误格式：

```

// 处理网络请求中的错误
private func handleError(_ error: AFError) -> NetworkRequestResult {
    if let underlyingError = error.underlyingError {
        let nsetError = underlyingError as NSError
        let code = nsetError.code
        // 判断常见的网络错误类型
        if code == NSURLErrorNotConnectedToInternet ||
            code == NSURLErrorTimedOut ||
            // 其他网络错误...
        {

```

```
// 给用户一个友好的提示
var userInfo = nerror.userInfo
userInfo[NSLocalizedStringKey] = "网络连接有问题喔～"
let currentError = NSError(domain: nerror.domain, code:
code, userInfo: userInfo)
return .failure(currentError)
}
}
return .failure(error)
}
```

代码说明

- 提取底层错误（如系统网络错误），判断错误类型（断网、超时等）；
- 替换错误描述为中文友好提示（如“网络连接有问题喔～”）；
- 返回处理后的错误，简化上层错误判断逻辑。

3. 解析 JSON 数据（BaseAPI）

将服务器返回的 JSON 数据，解析为指定模型对象（需遵循 Decodable 协议）：

代码说明

- 使用 Swift 自带的 JSONDecoder 解析数据，支持任意 Decodable 模型；
- 解析前打印原始数据（JSON 字符串），方便调试；
- 解析成功返回模型对象，失败返回“数据解析失败”的自定义错误。

五、为什么这样设计？

分层架构的核心优势的在于“解耦、复用、易维护”，具体如下：

1. **代码复用**：基础请求逻辑（如请求头、错误处理）仅写一次，所有业务模块可复用，减少重复代码；
2. **易于维护**：若需更换网络库（如从 Alamofire 改为 URLSession），仅修改 NetworkManager，上层代码无需变动；
3. **逻辑清晰**：UI 层专注“显示”，业务层专注“数据获取”，网络层专注“请求发送”，职责无重叠；
- 4.

扩展性好：新增业务模块（如“订单管理”）时，仅需创建 OrderApiObj 与 OrderVM，无需修改现有网络组件。

六、实际使用示例

以“获取用户列表”为例，展示项目中调用网络请求的完整代码：

```
// 创建一个API对象，指定业务模块（用户模块，API路径前缀 "user"）
let userApi = CommonApiObj<User>(moduleUrl: "user")
// 调用API获取列表数据（分页参数：第1页，每页10条）
userApi.list(parameters: ["page": 1, "pageSize": 10]) { result in
    switch result {
    case let .success(response):
        // 获取成功，处理数据（response为服务器返回的列表响应模型）
        if let userList = response.data?.records {
            // 更新UI显示用户列表（如赋值给@Published变量）
            self.userList = userList
        }
    case let .failure(error):
        // 获取失败，显示错误信息（如弹窗提示）
        print("获取用户列表失败：\(error.localizedDescription)")
        self.showErrorToast(message: error.localizedDescription)
    }
}
```

总结

项目网络请求架构通过“三层设计”实现了“高内聚、低耦合”，将复杂的网络操作拆解为简单模块，既简化了调用逻辑，又提升了代码可维护性与扩展性。无论是新增业务模块，还是更换网络库，都能以最小成本实现，非常适合中大型项目的长期迭代。

（注：文档部分内容可能由 AI 生成）